

tp_30-3

TP3 clase 30/3

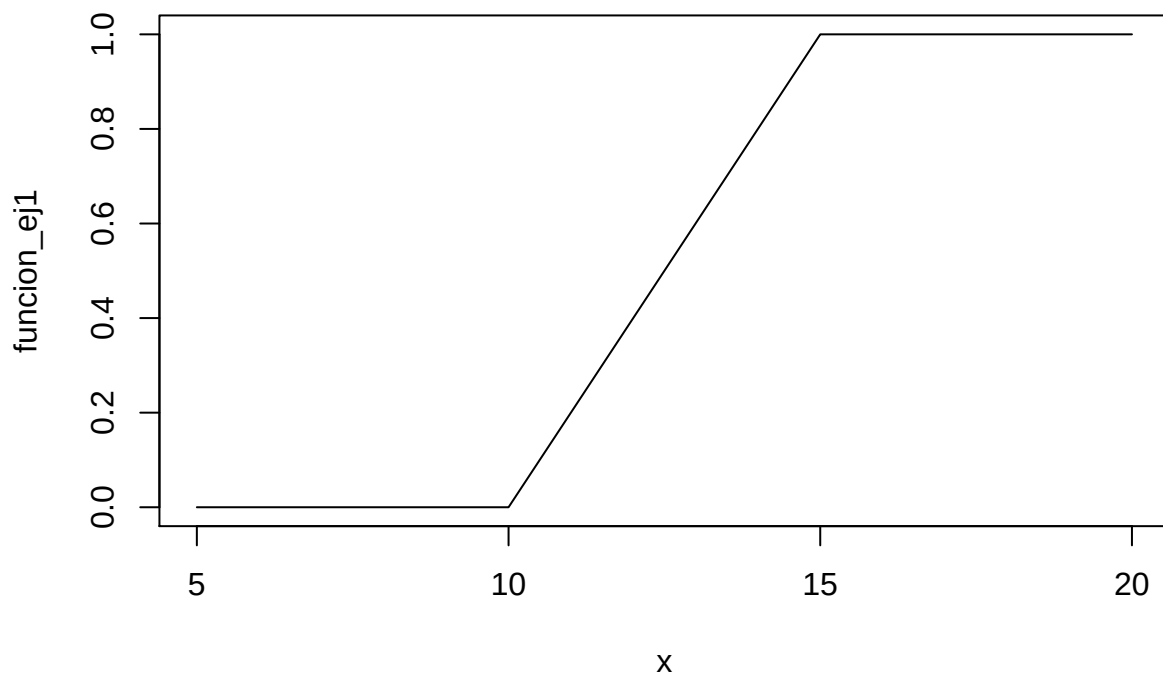
Tarea 1

Ejercicio 1

1. Implementar una funcion `funcion.h` que dado un número devuelva el valor de la función h en ese número.
2. Graficar la función $h(x)$ para $x \in [5, 20]$. Exporta tu gráfico y subilo a nuestro documento compartido que encontrarás en acá.
3. Si quisiera cambiar las constantes 10 y 15 por parámetros $a < b$, respectivamente, de tal manera que puedan ser fijados por el usuario, ¿como debería reprogramarse la `funcion.h`?

```
funcion_ej1 <-function(x,a,b)
{
  if(x<a){salida<-0}
  else if (x>=a & x<=b ){salida<-(x-10)/(15-10)}
  else {salida<-1}
  return(salida)
}

grilla_1<-seq(5,20,by=0.01)
yy_1<-lapply(grilla_1,funcion_ej1,10,15)
plot(grilla_1,yy_1, type="l",xlab="x",ylab="funcion_ej1")
```



Tarea 2: Combinemos bucles y condiciones.

1. Implementar una función `suma.positivos` que dado un vector `v` suma sus componente positivas; si no hay componentes positivas debe devolver el valor cero.

```
funcion_ej2 <-function(vect){
  suma = 0
  for (i in vect) {
    if(i>0){
      suma = suma + i}
    }
  return(suma)
}
vv<- c(10,1,-1,8)
funcion_ej2(vv)
```

```
## [1] 19
```

2. Calcular la suma de los números naturales positivos comprendidos entre los valores que le indicamos de manera personalizada en la hoja `suma positivos` que encontrarás acá y completar con el resultado obtenido en la columna D de la mencionada hoja de cálculo.

```
numeros <-seq(-636,3976,by=1)
funcion_ej2(numeros)
```

```
## [1] 7906276
```

```
sum(seq(1,3976,by=1))
```

```
## [1] 7906276
```

3. Implementar una función suma.si.hay.positivos que dado un vector v suma sus componente positivas; si no hay componentes positivas debe devolver un cartel que diga "No hay positivos".

```
#3
suma.si.hay.positivos <-function(vect){
  suma = 0
  for (i in vect) {
    if(i>0){
      suma = suma + i}
  }

  if (suma == 0){
    return("No hay positivos")
  }
  else {return(suma)}
}
vv <-seq(-636,-3976,by=-1)
suma.si.hay.positivos(vv)
```

```
## [1] "No hay positivos"
```

Tarea 3:

1 Crear una función suma.cubilete que simule arrojar el cubilete y dé por resultado la suma de las 5 caras obtenidas.

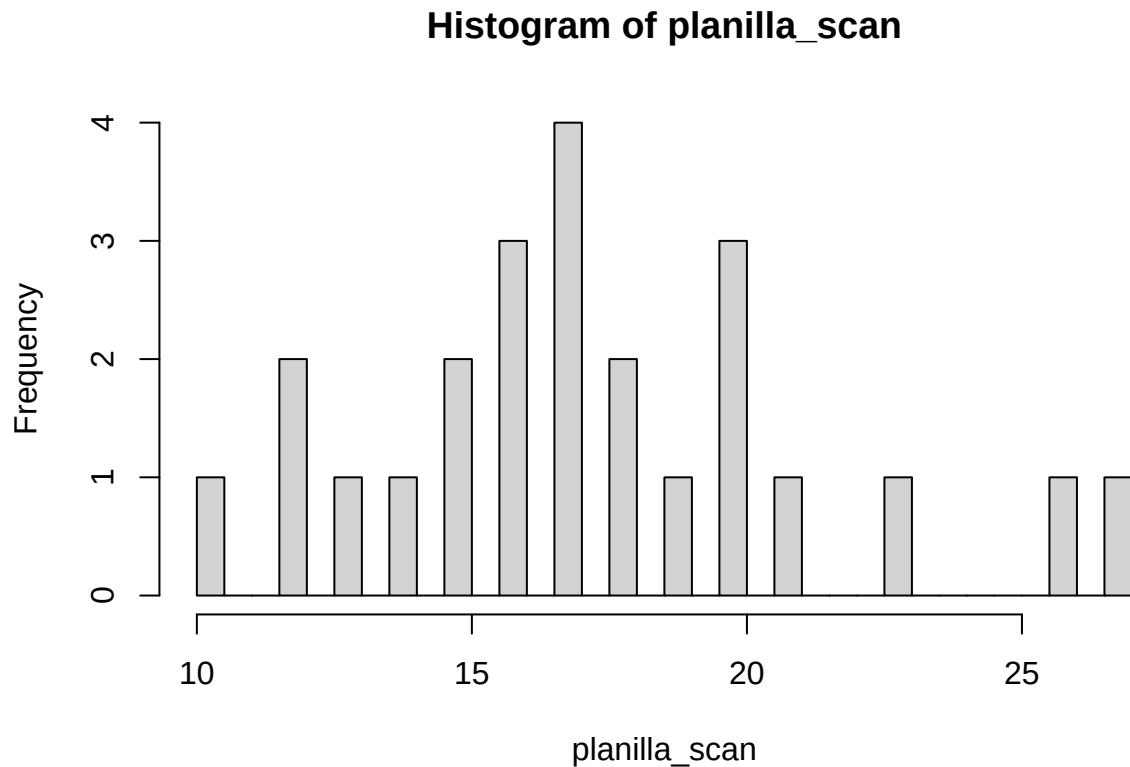
```
suma.cubilete <- function(n=5){
  return(sum(sample(1:6, size=n, replace=T)))
}
suma.cubilete()
```

```
## [1] 24
```

2,3,4

2. Invocar una vez la función suma.cubilete y completá la planilla aquí con el resultado obtenido.
3. Leemos la planilla y hacemos un histograma con los datos leídos.
4. Cada uno simula todo el proceso ahora con R: genera los 40 resultados del cubilete, calcula la suma y grafica un histograma con los resultados de su simulación.

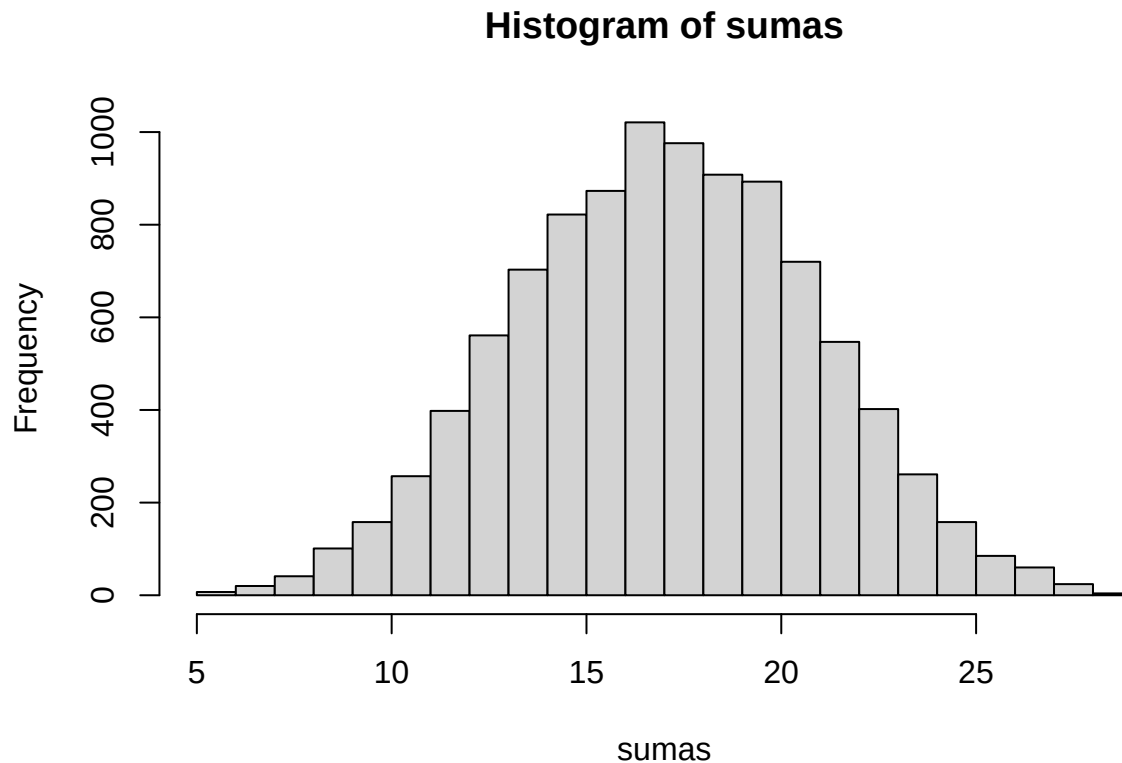
```
# setwd("/home/clinux01/Descargas")
# planilla <- read.csv('sumando.csv',dec=",", header=T)
# planilla_scan <- scan()
planilla_scan <- c(20,26,12,18,14,15,18,21,13,23,16,16,17,20,16,20,15,12,17,10,17,17,27,19)
hist(planilla_scan, breaks = 25)
```



5. Repetir en lugar de 40, 1000 veces el item anterior.

```
#set.seed(123)

tirar_muchos_datos <-function(n=40){
  sumas <-rep(NA, n) # sumas <-c()
  for (i in 1:n)
  {
    sumas[i] = suma.cubilete() # sumas <- c(sumas,suma.cubilete())
  }
  hist(sumas, breaks = 25)
}
tirar_muchos_datos(10000)
```



#6. * Implementar una funci3n todos.seis que cuenta la cantidad necesaria de lanzamientos hasta obtener el valor seis en todas la caras. ¿C3mo puede utilizar la funci3n suma cubilete para verificar si sale seis en todas las caras?

```
todos.seis <-function(){
  suma = 0
  todoseis=0
  while(todoseis==0){
    if(suma.cubilete() == 30)
    {todoseis=1}
    suma = suma + 1
  }
  return(suma)
}
todos.seis()
```

```
## [1] 6575
```

```
tirar_muchos_seis <-function(n=40){
  sumas <-rep(NA, n)
  for (i in 1:n)
  {
    sumas[i] = todos.seis()
  }
  hist(sumas, breaks = 50)
}
tirar_muchos_seis(100)
```

